

Aula 39 - Padrão MVC em PHP

Descrição: Esta aula aborda a arquitetura do padrão MVC em PHP e a reorganização estrutural das pastas de um projeto. Ensina o passo a passo prático para refatorar códigos "spaghetti" legados utilizando os princípios da Orientação a Objetos. Explora também o funcionamento do Front Controller, do roteamento centralizado e do conceito de autoloading via PSR-4.

Introdução:

O padrão MVC (Model-View-Controller) é a espinha dorsal da arquitetura de software para a web. Quando falamos de PHP antigo ou legado, é muito comum encontrar o famoso código "spaghetti" (onde HTML, CSS, SQL e lógica de negócios dividem alegremente a mesma linha de código). Migrar disso para o MVC não é só uma questão de capricho; é o que vai salvar sua sanidade mental, permitindo testabilidade, manutenção e escalabilidade.

Abaixo, um guia profundo de como funciona o MVC em PHP, como organizar a estrutura de pastas e um passo a passo prático de refatoração.

1. O Padrão MVC em Projetos PHP

O propósito do MVC é a Separação de Conceitos (**Separation of Concerns**). Cada camada tem um papel estrito e não deve assumir a responsabilidade das outras.

img01.png

Model (O Coração)

O Model gerencia os dados e as regras de negócio da aplicação. Ele não sabe que a web existe, não sabe o que é HTML e não lida com requisições HTTP.

- **O que faz:** Conecta ao banco de dados, valida dados de entrada segundo regras de negócio, executa queries (via PDO ou ORM) e manipula entidades.
- **Em PHP:** Geralmente são classes que representam tabelas ou serviços (ex: Usuario.php, ProdutoRepository.php).

View (A Vitrine)

A View é a interface apresentada ao usuário. Em aplicações PHP puras, é o local onde o HTML reside.

- **O que faz:** Recebe dados já processados e os exibe. Ela não faz requisições diretas ao banco de dados. Pode conter lógica simples de exibição (como um foreach para listar produtos ou um if para mostrar uma mensagem).
- **Em PHP:** Arquivos .php que são templates HTML com pequenas tags PHP (<?= \$variavel ?>) ou motores de renderização como Twig/Blade.

Controller (O Maestro)

O Controller intermedia a comunicação entre o Model e a View. Ele recebe o impacto da requisição do usuário.

- **O que faz:** Intercepta a requisição HTTP (via GET ou POST), pede os dados necessários ao Model, processa qualquer lógica de fluxo e repassa o resultado para a View correspondente.
- **Em PHP:** Classes com métodos (ações) chamados pelas rotas da aplicação (ex: HomeController.php,

2. Reorganizando o Projeto: Estrutura de Pastas

Para implementar o MVC de forma profissional, precisamos tirar os arquivos da raiz e organizá-los logicamente. Uma estrutura moderna e segura para PHP adota o padrão de expor apenas uma pasta pública (public/) para a internet, protegendo o código-fonte.

Sugestão de Arquitetura de Pastas

img02.png

O segredo: Front Controller e .htaccess

Em vez de acessar meu-projeto/usuarios/listar.php diretamente, o usuário acessa meu-projeto/usuarios. O arquivo .htaccess na raiz ou na pasta public/ redireciona todas as requisições para o public/index.php. Este arquivo lê a URL, descobre qual Controller deve chamar (Roteamento) e inicializa a aplicação.

3. Refatorando Código Spaghetti para MVC (Na Prática)

Vamos ver a anatomia de uma refatoração. Imagine o seguinte cenário clássico de código "tudo em um":

O Código Spaghetti (usuarios.php)

Este arquivo faz tudo: conecta ao banco, processa exclusão, busca dados e exibe HTML.

PHP

```
<?php
// TUDO MISTURADO: Banco, Lógica e HTML
$pdo = new PDO("mysql:host=localhost;dbname=teste", "root", "");

if (isset($_GET['excluir'])) {
    $id = $_GET['excluir'];
    $stmt = $pdo->prepare("DELETE FROM usuarios WHERE id = ?");
    $stmt->execute([$id]);
    header("Location: usuarios.php");
    exit;
}

$stmt = $pdo->query("SELECT * FROM usuarios");
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
?>
<!DOCTYPE html>
<html>
<head><title>Lista de Usuários</title></head>
<body>
    <h1>Usuários</h1>
    <ul>
        <?php foreach($usuarios as $u): ?>
```

```

        <li>
            <?= $u['nome'] ?>
            <a href="usuarios.php?excluir=<?= $u['id'] ?>">Excluir</a>
        </li>
    <?php endforeach; ?>
</ul>
</body>
</html>

```

Aplicando a Separação (Refatorado para MVC)

Para organizar isso, vamos fatiar esse arquivo nas pastas corretas usando orientação a objetos e boas práticas (como injeção de dependência simplificada).

1. A Camada de Configuração (app/config/database.php)

Isolamos a conexão com o banco de dados.

PHP

```

<?php
namespace App\Config;

use PDO;

class Database {
    public static function getConnection() {
        // Em produção, use variáveis de ambiente (.env)
        return new PDO("mysql:host=localhost;dbname=teste", "root", "");
    }
}

```

2. O Model (app/models/Usuario.php)

Toda a interação com a tabela usuarios e suas regras ficam aqui. O Model não sabe quem o chamou, ele apenas entrega ou manipula dados.

PHP

```

<?php
namespace App\Models;

use PDO;
use App\Config\Database;

class Usuario {
    private $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function getAll() {

```

```

$stmt = $this->db->query("SELECT * FROM usuarios");
return $stmt->fetchAll(PDO::FETCH_ASSOC);
}

public function delete($id) {
    $stmt = $this->db->prepare("DELETE FROM usuarios WHERE id = ?");
    return $stmt->execute([$id]);
}
}

```

3. O Controller (*app/controllers/UsuarioController.php*)

Ele toma as decisões. Recebe a ação, fala com o Model e decide qual View carregar.

PHP

```

<?php
namespace App\Controllers;

use App\Models\Usuario;

class UsuarioController {

    // Ação de Listar
    public function index() {
        $usuarioModel = new Usuario();
        $usuarios = $usuarioModel->getAll();

        // Passa os dados para a View
        require_once __DIR__ . '/../views/usuarios/listar.php';
    }

    // Ação de Excluir
    public function excluir() {
        if (isset($_GET['id'])) {
            $usuarioModel = new Usuario();
            $usuarioModel->delete($_GET['id']);
        }
        // Redireciona usando a rota interna
        header("Location: /usuarios");
        exit;
    }
}
}

```

4. A View (*app/views/usuarios/listar.php*)

Código HTML limpo. Sem conexões SQL, sem queries. Apenas lê as variáveis que o Controller disponibilizou (neste caso, \$usuarios).

PHP

```

<!DOCTYPE html>

```

```
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Lista de Usuários</title>
</head>
<body>
  <h1>Usuários</h1>
  <ul>
    <?php foreach($usuarios as $u): ?>
      <li>
        <? = htmlspecialchars($u['nome']) ?>
        <a href="/usuarios/excluir?id=<? = $u['id'] ?>">Excluir</a>
      </li>
    <?php endforeach; ?>
  </ul>
</body>
</html>
```

Conectando tudo: O Roteador Simplificado (public/index.php)

Para que tudo isso funcione sem um framework pesado (como Laravel ou Symfony), o seu arquivo central index.php precisa interpretar a URL e chamar o Controller certo. Um exemplo bem básico de como isso funciona:

```
PHP
<?php
// Autoload para carregar as classes automaticamente (PSR-4 via Composer ou manual)
require_once __DIR__ . '/../vendor/autoload.php';

// Captura a URL digitada (ex: /usuarios ou /usuarios/excluir)
$url = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);

// Roteamento simples
if ($url === '/usuarios' || $url === '/') {
    $controller = new \App\Controllers\UsuarioController();
    $controller->index();
} elseif ($url === '/usuarios/excluir') {
    $controller = new \App\Controllers\UsuarioController();
    $controller->excluir();
} else {
    http_response_code(404);
    echo "Página não encontrada!";
}
```

Benefícios Obtidos com a Refatoração

Aspecto	Antes (Spaghetti)	Depois (MVC Profissional)
Manutenção	Alterar o layout corria o risco de	Layout (View) e Queries (Model) estão em

	quebrar uma query SQL.	arquivos totalmente isolados.
Reutilização	Se precisasse listar usuários em uma API, teria que reescrever a query.	Basta chamar o método <code>\$usuarioModel->getAll()</code> em um novo ApiController.
Segurança	Conexões e credenciais espalhadas por múltiplos arquivos.	Centralizado em <code>/config</code> , facilitando o uso de variáveis de ambiente seguras.
Trabalho em Equipe	Desenvolvedores front-end e back-end mexiam no mesmo arquivo, gerando conflitos.	Front-end mexe apenas na pasta <code>/views</code> , Back-end foca em <code>/models</code> e <code>/controllers</code> .

Resumindo:

Aqui estão os pilares fundamentais e inegociáveis extraídos dessa análise para guiar a reestruturação do seu projeto PHP:

Os Pilares Fundamentais do MVC em PHP

1. O "Mantra" da Separação de Responsabilidades

- **O Model é agnóstico à Web:** Ele nunca deve conter códigos como `echo`, tags HTML ou acessar diretamente superglobais como `$_POST` ou `$_GET`. O Model lida estritamente com dados e regras de negócio.
- **A View é "burra" (por design):** Ela não sabe o que é um banco de dados, não faz conexões e não executa queries. Ela é apenas um template que recebe variáveis prontas do Controller e as exibe de forma limpa.
- **O Controller é um intermediário magro:** Ele não deve conter queries SQL complexas nem lógica de negócios pesada. Sua única função é receber a requisição do usuário, acionar o Model correto e despachar o resultado para a View adequada.

2. Segurança e a Arquitetura de "Front Controller"

- **Exposição Mínima:** A raiz do seu servidor web deve apontar **apenas** para a pasta `public/`.
- O arquivo `public/index.php` centraliza todas as requisições (com a ajuda do `.htaccess`). Isso impede que usuários naveguem pela estrutura de pastas do sistema e acessem arquivos de script (`.php`) diretamente, blindando o código-fonte contra invasões e vazamento de credenciais.

3. A Anatomia da Refatoração (O "Fatiamento")

- Migrar do código *spaghetti* para o MVC exige abandonar scripts puramente lineares e adotar a **Orientação a Objetos (POO)**.
- A transição bem-sucedida segue uma ordem lógica:
 1. Centralizar e isolar a conexão com o banco (`config/`).
 2. Extrair e isolar as queries em métodos de classes (`models/`).
 3. Isolar o HTML puro em arquivos de template (`views/`).
 4. Unir os pontos usando métodos de controle (`controllers/`).

4. O Retorno sobre o Investimento (ROI) Técnico

- **Desacoplamento:** Alterar o design visual da aplicação (CSS/HTML) tem **zero risco** de quebrar uma query no banco de dados.

- **Preparado para o Futuro:** Se amanhã você precisar criar um aplicativo mobile ou integrar com outro sistema, o seu **Model** já está pronto para ser reutilizado por um controller de API que retorna JSON, sem que você precise reescrever uma única linha de SQL.
-

PONTO DE ATENÇÃO

Se existe um ponto nessa transição que exige **total atenção** e merece uma pesquisa profunda, ele é: **O Roteamento Centralizado e o padrão Front Controller (com Autoloading PSR-4)**.

Por que este ponto? Porque ele é a **porta de entrada** e a fundação técnica de todo o ecossistema MVC. Se você não dominar o roteamento, você não conseguirá fazer o MVC funcionar na prática, e seu projeto continuará refém de arquivos soltos acessados diretamente pelo navegador.

O que pesquisar a fundo neste tópico?

Para dominar o coração da infraestrutura de um framework MVC, divida sua pesquisa em três pilares técnicos:

1. Reescrevendo URLs com .htaccess (ou Nginx try_files)

No código antigo, para ver os usuários, o link era `meusite.com/usuarios.php`. No MVC profissional, o link deve ser `meusite.com/usuarios`.

- **O foco da pesquisa:** Como configurar o servidor web para que **qualquer** URL digitada (seja `/produtos`, `/perfil` ou `/usuarios/excluir`) seja capturada internamente e enviada de forma invisível para o arquivo `public/index.php`.

2. A Arquitetura do Roteador (Router)

O `index.php` recebe a string da URL (ex: `/usuarios/editar/5`). O Roteador é o mecanismo que quebra essa string para descobrir:

1. Qual **Classe Controller** instanciar (`UsuarioController`).
2. Qual **Método** executar (`editar`).
3. Quais **Parâmetros** passar para esse método (`5`).
 - **O foco da pesquisa:** Padrões de projeto para criação de roteadores simples em PHP, ou como integrar um componente de rotas profissional e seguro (como o `Symfony Router` ou `FastRoute`) usando o Composer.

3. Autoloading de Classes com PSR-4

No modelo antigo, enche-se o código de `include` e `require` no topo de cada página. No MVC, com dezenas de Models e Controllers, gerenciar isso manualmente se torna um inferno.

- **O foco da pesquisa:** A norma **PSR-4** (padrão da comunidade PHP para carregamento automático de classes baseado em Namespaces) e como o **Composer** automatiza isso com o comando `composer dump-autoload`.

Por que este ponto é o mais crítico?

- **Segurança Absoluta:** É o Front Controller que garante que nenhum arquivo do seu sistema (além do `index.php` e dos assets como CSS/JS) possa ser executado diretamente pelo navegador. Se alguém tentar acessar `meusite.com/app/config/database.php`, o servidor web vai negar ou o roteador vai disparar um Erro 404.

- **É o divisor de águas:** Entender o Model, a View e o Controller isoladamente é relativamente simples (são apenas classes e HTML). O verdadeiro desafio do desenvolvedor PHP que está saindo do nível júnior/pleno para o sênior é **compreender o motor que conecta essas três partes**.

Dominando o Roteamento e o Front Controller, você não estará apenas organizando pastas; você estará criando, do zero, a estrutura de um **framework PHP proprietário**, exatamente da mesma forma que gigantes como o Laravel e o Symfony operam por baixo do capô.